UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento: Departamento de electronica y telecomunicaciones

Asignatura: Fundamentos de programación

NRC: 36466

Apellidos y Nombres: Campo Quinga Rober Ethereen

Deber: Ejercicios de programación y algoritmia

Problema 1.1. Media de dos valores

Desarolla un programa que ingrese dos números por teclado e imprima su media

Realiza y completa la tabla de objetos del problema 1.1:

1. Analisis del problema.

Idea general: Se piden dos numeros y se calcula el valor que queda justo en medio de ambos: se suma dividida entre 2

Entradas. Dos numeros reales, a y b

Proceso. Se suman a y b y el resultado se divide entre 2: media = (a + b) / 2.

Salida: Un mensaje con la media de a y b

Restricción: ninguna, los numeros pueden ser positivos, negativos o cero.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (primer número)	a	var	real
Dato de entrada (segundo número)	b	var	real
Resultado (media)	media	var	real
Divisor de la media	2	cte	entero

3. Seudocodigo

```
Inicio
    Leer a, b
    media ← (a + b) / 2
    Escribir "La media de ", a, " y ", b, " es", media
Fin
```

ESTILO

Escaneado con CamScanner

4. Prueba de escritorio

Caso	a	b	a + b	media
Normal	4	6	10	5
Normal (decimal)	3	8	11	5,5
Negativos	-5	-3	-8	-4
Cero (a)	-2	2	0	0
Decimales	1,5	2,5	4	2
Limite (ceros)	0	0	0	0

Problema 1.2. Valor absoluto de x al cubo

Desarrolla un programa que lea un número por teclado y escriba por pantalla $|x|^3$

1. Análisis del problema

Idea general: El valor absoluto le quita el signo al número, luego ese valor se eleva al cubo, por lo que el resultado nunca es negativo

Entradas: Un numero real x

Proceso: si $x < 0$, el valor absoluto es $-x$; si no, es el propio x. se multiplica ese valor tres veces por si mismo: $|x|^3 = abs\,x \cdot abs\,x \cdot abs\,x$.

Salida: El valor de $|x|^3$

Restricción: ninguna. Ejemplo: $x = -3 \rightarrow |-3| = 3 \Rightarrow 3^3 = 27$

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	x	var	real
Valor absoluto de x	absx	var	real
Resultado $|x|^3$	resultado	var	real
Cero (signo)	0	cte	entero

3. Seudocodigo.

```
Inicio
    Leer x
    Si x < 0 entonces
        absx ← -x
    Sino
        absx ← x
    Fin Si
    resultado ← absx * absx * absx
    Escribir "|", x, "|^3 =", resultado
Fin.
```

Escaneado con CamScanner

4. Prueba de escritorio

Caso	x	x < 0	abs x	resultado
Normal (positivo)	2	NO	2	8
Normal (negativo)	-3	Si	3	27
Decimal	1,5	NO	1,5	3,375
Decimal negativo	-0,5	Si	0,5	0,125
Limite (cero)	0	No	0	0

Problema 1.3. Divisible

Desarrolla un programa que lea dos números enteros por teclado y determine si: el primero es divisible por el segundo. Se mostrara por pantalla el resultado.

1. Analisis del problema

Entradas: Dos enteros, el dividendo x y el divisor y

Proceso: x es divisible por y cuando el resto de la división entera es cero (x % y = 0)

Salida: Un mensaje que indica si x es o no divisible por y.

Restricción: Y no puede ser 0, por que no existe la división entre cero.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (dividendo)	x	var	entero
Dato de entrada (divisor)	y	var	entero
Cero	0	cte	entero

3. Seudocodigo

```
Inicio
    Leer x, y
    Si y = 0 entonces
        Escribir "No es posible dividir entre cero"
    Sino
        Si x mod y = 0 entonces
            Escribir x, " es divisible por ", y
            Escribir x, " no es divisible por ", y
        Fin si
    Fin
```

4. Prueba de escritorio

ESTILO

Escaneado con CamScanner

Caso	x	y	y=0	x mod y	Salida
Normal	12	4	No	0	12 es divisible por 4
Normal	7	2	No	1	7 no es divisible por 2
Divisor igual	5	5	No	0	5 es divisible por 5
Límite (dividendo 0)	0	3	No	0	0 es divisible por 3
Negativo	-9	3	No	0	-9 es divisible por 3
Negativo	7	-2	No	1	7 no es divisible por -2
Error	8	0	Si	-	No es posible dividir entre cero

Problema 1.4. Intervalo

Desarrolla un programa que lea un numero por teclado y determine si pertenece al intervalo [0,10], indicando el resultado por pantalla.

1. Análisis del problema

Idea general: Un número está en el intervalo cerrado [0,10] si no es menor que 0 y no es mayor que 10; es decir, cuando se cumplen los dos condiciones al mismo tiempo.

Entradas: Un numero real x

Proceso: Se evalúan $x \geq 0$ y $x \leq 10$ unidas con el operador lógico Y (&& en C). Solo si ambas son verdaderas, x pertenece al intervalo. No se escribe $0 \leq x \leq 10$ en una sola condición por que en C se evalua de izquierda a derecha

Salida: Un mensaje que indica si x pertenece o no al intervalo [0,10]

Restricción: Ninguna; los extremos 0 y 10 si pertenecen por que el intervalo es cerrado.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	x	var	real
límite inferior del intervalo	0	cte	entero
límite superior del intervalo	10	cte	entero

3. Seudocódigo

```
Inicio
    Leer x
    Si x >= 0 y x <= 10 entonces
        Escribir x, " pertenece al intervalo [0, 10]"
    Sino
        Escribir x, " no pertenece al intervalo [0, 10]
    Fin Si
Fin
```

Escaneado con CamScanner

4. Prueba de escritorio

Caso	x	x ≥ 0	x ≤ 10	y	Salida
Normal (dentro)	5	Si	Si	Si	5 pertenece al intervalo [0,10]
Normal (decimal)	7,25	Si	Si	Si	7,25 pertenece al intervalo [0,10]
Límite inferior	0	Si	Si	Si	0 pertenece al intervalo [0,10]
Límite superior	10	Si	Si	Si	10 pertenece al intervalo [0,10]
Fuera (mayor)	15	Si	No	No	15 no pertenece al intervalo [0,10]
Fuera (menor)	-0,1	No	Si	No	-0,1 no pertenece al intervalo [0,10]

Problema 1.5. Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c, interpretándolo como el número de segundos que dura un evento, y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa.

1. Análisis del problema.

Idea general: Se convierte una cantidad de segundos en una duración «normal»: primero se sacan los días completos, con lo que sobra sacan las horas completas, luego los minutos, y lo último que queda son los segundos.

Entradas: Un entero c, la cantidad de segundos

Proceso: dia = c div 86400; resto = c mod 86400; horas = resto div 3600; resto mod 3600; mintos = resto div 60; segundos = resto mod 60; Aquí 86400 = 24·60·60 y 3600 = 60·60

Salida. Un mensaje con los días, horas, minutos y segundos equivalentes.

Restricción: C no puede ser negativo (una duración no puede ser negativa)

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (segundos)	C	var	entero
Días completos	días	var	entero
Horas completas	horas	var	entero
Min completos	minutos	var	entero
S sobrantes	segundos	var	entero
S que quedan por convertir	resto	var	entero
S en un día (24·60·60)	S_dia = 84600	cte	entero
S en una hora (60·60)	S_Hora = 3600	cte	entero
S en un minuto	S_Min = 60	cte	entero
Cero (para validar)	0	cte	entero

ESTILO

Escaneado con CamScanner

3. Seudocodigo

```
Inicio
    Leer c
    Si c < 0 entonces
        Escribir "los segundos no pueden ser negativos"
    Sino
        dias ← c div 86400
        resto ← c mod 86400
        horas ← resto div 3600
        resto ← resto mod 3600
        minutos ← resto div 60
        segundos ← resto mod 60
        Escribir c, " segundos = ", dias, " dias, ", horas, " horas "
                 minutos, " minutos y ", segundos " segundos."
    Fin Si
Fin
```

4. Prueba de escritorio

Caso	c	c<0	dias	horas	minutos	segundos
Normal	10000	No	0	2	46	40
Todos los valores	90061	No	1	1	1	1
Menos de un minuto	59	No	0	0	0	59
Exactamente 1 hora	3600	No	0	1	0	0
Exactamente 1 dia	86400	No	1	0	0	0
Limite (cero)	0	No	0	0	0	0
Error	-5	Si	-	-	-	-

Problema 1.6. Maximo, minimo y media de alturas

Desarrolla un programa que lea por teclado un numero indeterminado de alturas y deje de leer cuando se introduzcan una negativa. El programa debe calcular e imprimir la media, el maximo y el minimo de las alturas introducidas.

1. Analisis del problema

Idea general: Se leen alturas una por una sin saber cuántas serán; el ingreso de un numero negativo es la señal para parar (centinela). Mientras se leen, se lleva la suma, la cuenta, la mayor y la menor altura, sin guardar todos

Entradas: Alturas h (reales), hasta que se ingrese una negativa

Proceso: Por cada altura valida ($h \geq 0$): se suma a suma, se aumenta cant y se actualizan maximo y minimo (la primera altura inicializa ambos).

Salida: La media, el maximo y el minimo de las alturas.

Restricción: La altura negativa solo termina la lectura; no cuenta como dato. Si la primera altura ya es negativa no hay datos y se informa, lo que evita dividir entre cero

Escaneado con CamScanner

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de altura (entrada)	h	var	real
Suma de las alturas	suma	var	real
Cantidad de alturas leidas	cont	var	entero
Altura maxima	maximo	var	real
Altura minima	minimo	var	real
Media de las alturas	media	var	real
Cero	0	cte	entero

3. Seudocodigo

```
Inicio
    suma ← 0
    cont ← 0
    Leer h
    Mientras h >= 0 hacer
        si cont = 0 entonces
            maximo ← h
            minimo ← h
        Sino
            Si h > maximo entonces
                maximo ← h
            Fin Si
            Si h < minimo entonces
                minimo ← h
            Fin Si
        Fin Si
        suma ← suma + h
        cont ← cont + 1
        leer h
    Fin Mientras
    Si cont = 0 entonces
        Escribir "No se ingresaron alturas"
    Sino
        media ← suma / cont
        Escribir "Media: ", media
        Escribir "Minimo: ", minimo
        Escribir "Maximo: ", maximo
    Fin Si
Fin
```

ESTILO

Escaneado con CamScanner

4. Prueba de escritorio

Caso	Alturas ingresadas	cont	Media	Maximo	Minimo
Normal	1.70, 1.65, 1.80, -1	3	1,72	1,80	1,65
Normal	1.60, 1.90, 1.75, 1.55, -2	4	1,70	1.90	1,55
Limite (una sola altura)	1.50, -1	1	1,50	1.50	1,50
Error (sin datos)	-1	0	No se ingresó	-	-

Problema 1.7. Raiz cuadrada entera

Desarrolla un programa que calcule la raiz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero $R > 0$ tal que $R^2 \leq N$

1 Analisis del problema

Idea general: La raiz cuadrada entera por defecto de N es el entero más grande cuyo cuadrado no supera a N. Se prueba $i = 1, 2, 3, \ldots$ hasta que i^2 se pasa de N; el ultimo valor que no se pasaba es la respuesta.

Entradas. Un numero entero positivo N

Proceso. Se empieca con $i = 1, 2$. mientras $i^2 \leq N$, se suma 1 a i, 3 cuando $i^2 > N$ el ciclo termina; como i ya se pasó de la raiz, $R = i - 1$

Salida: Un mensaje con la raiz cuadrada entera R

Restricción. N debe ser mayor que 0 se valida antes de iniciar el ciclo. Ejemplo: $N = 10 \rightarrow i = 1, 2, 3$ cumplen; $i = 4$ no $(16 > 10) \rightarrow R = 3$

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (numero)	N	var	entero
Contador de prueba	i	var	entero largo
Raiz entera por defecto	R	var	entero largo
Valor inicial del contador	1	cte	entero
Cero (para validar)	0	cte	entero

3. Seudocodigo

```
Inicio
    Leer N
    Si N <= 0 entonces
        Escribir "N debe ser positivo"
    Sino
        i ← 1
        Mientras i * i <= N hacer
            i ← i + 1
        Fin mientras
        R ← i - 1
        Escribir "La raiz cuadrada entera de ", N, "es", R
    FinSi
Fin
```

Escaneado con CamScanner

4. Prueba de escritorio

Caso	N	N ≤ 0	i final (i² > N)	R = i − 1	Salida
Normal	10	No	4	3	La raíz cuadrada entera de 10 es 3
Normal	26	No	6	5	La raíz cuadrada entera de 26 es 5
Cuadrado perfecto	16	No	5	4	La raíz cuadrada entera de 16 es 4
Límite	1	No	2	1	La raíz cuadrada entera de 1 es 1
Límite (no cuadrado)	2	No	2	1	La raíz cuadrada entera de 2 es 1
Error	0	Sí	-	-	N debe ser positivo
Error	-4	Sí	-	-	N debe ser positivo

Traza del caso N = 10

i	i * i	i * i ≤ N	Acción
1	1	Sí	i ← 2
2	4	Sí	i ← 3
3	9	Sí	i ← 4
4	16	No	Fin del ciclo; R = 4 − 1 = 3

Problema 1.8. Número primo

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

1 Análisis del problema

Idea general: Un número es primo si solo es divisible entre 1 y entre sí mismo. Se intenta dividirlo entre 2, 3, 4...; si alguno lo divide exactamente, ya no es primo.

Entradas: Un entero n > 1

Proceso: se prueban los divisores d = 2, 3, mientras d · d ≤ n (basta llegar hasta √n: si n tuviera un divisor mayor que √n, tendría también uno menor que ya se habría probado). Si algún d cumple n mod d = 0, n no es primo

Salida: Un mensaje que indica si n es primo o no

Restricción: n > 1; el 1, el 0 y los negativos no son primos y se rechazan con un mensaje.

2. Tabla de objetos:

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Divisor de prueba	d	var	entero
Indicador de primalidad	esPrimo	var	lógico
Primer divisor a probar	2	cte	entero
Límite de validación	1	cte	entero

ESTILO

Escaneado con CamScanner

3. Seudocodigo.

```
Inicio
  Leer n
  Si n <= 1 entonces
    Escribir "El número debe ser mayor que 1"
  Sino
  esPrimo <- Verdadero
    d <- 2
  Mientras d * d <= n y esPrimo hacer
    Si n mod d = 0 entonces
      esPrimo <- Falso
    Fin Si
    d <- d + 1
  Fin Mientras
  Si esPrimo entonces
    Escribir n, " es primo"
  Sino
    Escribir n, " no es primo"

  Fin Si
Fin
```

4. Prueba de escritorio

Caso	n	n ≤ 1	Divisores probados (d)	esPrimo	Salida
Normal (primo)	17	No	2, 3, 4	Verdadero	17 es primo
Normal (compuesto)	9	No	2, 3	Falso	9 no es primo
Compuesto par	10	No	2	Falso	10 no es primo
Cuadrado de un primo	25	No	2, 3, 4, 5	Falso	25 no es primo
Límite (menor primo)	2	No	ninguno (2*2 > 2)	Verdadero	2 es primo
Límite	3	No	ninguno (2*2 > 3)	Verdadero	3 es primo
Error	1	Si	–	–	Numero > 1
Error	-7	Si	–	–	Numero > 1

Problema 1.9. Numeros primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los numeros primos comprendidos entre dos numeros enteros introducidos por teclado.

1. Analisis del problema

Idea general: Es el problema 1.8 repetido para cada numero del rango: se recorre cada n desde el menor hasta el mayor y se muestran los que resulten primos.

Entradas: dos enteros a y b (en cualquier orden)

Proceso: si a > b se intercambian. si a < 2 se sube a 2, por que no hay primos menores, para cada n de a hasta b se aplica la prueba de primalidad del 1.8, si n es primo se muestra y se cuenta.

Salida: La lista de primos del rango (incluidos los extremos) o un mensaje si no hay ninguno

Restricción ninguna sobre el orden o el signo de los datos; el programa los corrige.

Escaneado con CamScanner

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (primer extremo)	a	var	Entero
Dato de entrada (segundo extremo)	b	var	Entero
Auxiliar para el intercambio	aux	var	Entero
Numero que se esta evaluando	n	var	Entero
Divisor de prueba	d	var	Entero
Indicador de primalidad	esPrimo	var	logico
Cantidad de primos encontrados	cant	var	Entero
Menor numero primo	2	cte	Entero

3. Seudocodigo

```
Inicio
   Leer a, b
   Si a > b entonces
      aux ← a
      a ← b
      b ← aux
Fin Si
Si a < 2 entonces
    a ← 2
Fin si
Cant ← 0
n ← a
Mientras n <= b hacer
   es Primo ← Verdadero
   d ← 2
 Mientras d * d <= n Y esPrimo hacer
      Sin n mod d = 0 entonces
            es primo ← Falso
      Fin Si
      d ← d + 1
      Fin Mientras
      Si es Primo entonces
         Escribir n
         cont ← cont + 1
      Fin Mientras
      Si cont = 0 entonces
           Escribir "No hay numeros primos en el rango"
      Fin Si
Fin
```

ESTILO

Escaneado con CamScanner

4. Prueba de escritorio

Caso	a	b	Rango revisado	Salida
Normal	10	30	10 a 30	11, 13, 17, 19, 23, 29
Datos invertidos	30	10	10 a 30 (se intercambian)	11, 13, 17, 19, 23, 29
Desde 1	1	10	2 a 10 (a se sube a 2)	2 3 5 7
Un solo valor	7	7	7 a 7	7
Con negativos	-5	3	2 a 3	2 3
Sin primos	14	16	14 a 16	No hay en el rango
Límite	0	1	2 a 1 (rango vacío)	No hay en el rango

Problema 1.10. Factorial

Desarrolla un programa que lea un número entero $n \geq 0$ y calcule $n! = 1 \cdot 2 \cdot \ldots \cdot n$, sabiendo que $0! = 1$

1. Análisis del problema

Idea general: n! es el producto de todos los enteros de 1 hasta n. Se parte de un acumulador en 1 y en cada vuelta se multiplica por el siguiente entero.

Entrada: Un entero n

Proceso: fact ← 1 para i = 1, 2, ..., n : fact ← * i. Si n = 0 el ciclo no se ejecuta y queda 0! = 1, que es lo correcto

Salida: El valor de n!

Restricción: $n \geq 0$ (el factorial de un negativo no existe) y $n \leq 20$ (20! es el mayor factorial que cabe en un entero de 64 bits)

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Acumulador del producto	fact	var	entero largo
Contador del ciclo	i	var	entero
Máximo n permitido	20	cte	entero
Cero (para validar)	0	cte	entero

3. Seudocódigo

```
Inicio
    Leer n
    Si n < 0 entonces
        Escribir "El factorial no está definido para negativos"
    Si no
        Si n > 20 entonces
            Escribir "n es demasiado grande (máximo 20)
        Si no
            fact ← 1
            i ← 1
            Mientras i <= n hacer
            fact ← fact * i
            i ← i + 1
            Fin mientras
            Escribir n, "! = ", fact
Fin
```

Escaneado con CamScanner

4. Prueba de escritorio

Caso	n	n<0	n>20	fact	Salida
Normal	5	No	No	120	5! = 120
Normal	10	No	No	3628800	10! = 3628800
Límite (n=0)	0	No	No	1	0! = 1
Límite (n=1)	1	No	No	1	1! = 1
Límite (máximo)	20	No	No	2432902008176640000	20! = 2432902008176640000
Error	-3	Si	-	-	Factorial no negativos
Error	21	No	Si	-	n demasiado grande (max 20)

Para pensar: Que ocurre si el usuario ingresa un número negativo?

El Factorial no esta definido para los negativos. Sin validación, la condición $i \leq n$ seria falsa desde el inicio, el ciclo no se ejecutaria y el programa mostraria 1, un resultado incorrecto sin ningún aviso. Por eso se valido $n < 0$ y se muestra un mensaje de error. Ejemplo: n = -3 -> Factorial no definido para negativos.

Y con n = 20 en una variable de tipo int?

Un int de 32 bits solo llega hasta 2 147 483 647. Como 12! = 479 001 600 todavia cabe, pero 13! = 6 227 020 800 ya no, con n = 20 el resultado se desborda y sale un numero equivocado.

Escaneado con CamScanner